UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

CÁTEDRA
DISEÑO DE SISTEMAS
DE INFORMACIÓN

MetaMapa: Sistema de gestión de mapeos colaborativos

Trabajo Práctico Anual Integrador
-2025-

ENTREGA 3: Integración, y Modelado en Objetos - Parte III

Objetivos de la entrega

- Diseñar e implementar, de manera incremental, las nuevas funcionalidades.
- Incorporar flujos de trabajo asincrónicos.
- Enfrentar decisiones de arquitectura.

Alcance

- Servicio y componente agregador.
- Agregación de fuentes y algoritmos de consenso.
- Implementación de tareas asíncronas mediante programación externa (tareas programadas estilo cron).

Unidades del Programa Vinculadas

- Unidad 2: Herramientas de Concepción y Comunicación del Diseño
- Unidad 3: Diseño con Objetos
- Unidad 6: Diseño de Arquitectura
- Unidad 8: Validación del Diseño

Dominio

Servicio de agregación

Para esta iteración, se requiere modelar el servicio de agregación descripto en el diagrama de despliegue inicial. Esto permitirá que las personas visitantes y/o contribuyentes de la plataforma accedan a hechos provenientes de una o más fuentes.

Al igual que con el resto de las fuentes desarrolladas en entregas pasadas, el servicio de agregación podrá ser desplegado de dos maneras:

- como un nodo independiente, que solo brindará funciones de agregación;
- o como un componente reutilizable, que podes ser empleado en cualquier otro modo del sistema y combinado con otras fuentes y/o colecciones.

La incorporación de los agregadores permitirá que **las colecciones** contengan hechos de distintas fuentes, sean estas estáticas, dinámicas o provenientes de servicios externos (fuentes proxy).

Costo de las operaciones

Sin embargo, se ha identificado un posible problema: dado que ahora las colecciones vinculadas a una fuente agregadora podría estar internamente realizando consultas a múltiples sistemas externos cada vez se realiza una búsqueda, este tipo de operaciones podrían ser costosas en términos de ciclos de cómputo y/o uso de memoria.

Por ese motivo se pide también que se diseñe al componente agregador de forma que éste pueda contar con **copias locales** de los hechos, **actualizadas periódicamente**, de forma similar a lo realizado con las fuentes proxy en la entrega anterior.

Sobre este requerimiento no contamos con una descripción detallada, sino que queda a criterio de nuestro equipo cómo encarar este problema, qué hechos copiar localmente y cuándo hacerlo. Es importante **maximizar la reutilización de lógica** y contamos con vía libre para realizar los refactors que consideremos necesarios.

Modos de navegación

En iteraciones anteriores, cualquier persona podía navegar las colecciones, aplicando opcionalmente filtros. Sin embargo, en esta entrega se incorpora el concepto de ***modo de navegación***, que le permitirá elegir qué hechos se mostrarán**:**

- de forma ***irrestricta***, en la cual navegarán todos los hechos, sin ningún tipo de curación;
- de forma ***curada***, en la cual se navegarán sólo los hechos para los que se tiene ***consenso***.

Consenso

Para reducir la desconfianza que podría darse por presentar hechos dudosos o falsos, al crear una colección, se podrá especificar opcionalmente un algoritmo de ***consenso***. Este algoritmo servirá para ponderar el grado de confianza que hay en un hecho, en base a cuántas veces se lo ha visto a lo largo de las distintas fuentes **del nodo**.

Estos son algunos de los algoritmos que se implementarán:

- múltiples menciones: si al menos dos fuentes **del nodo** contienen un mismo hecho y ninguna otra fuente **del nodo** contiene otro de igual título pero diferentes atributos, se lo considera consensuado;
- mayoría simple: si al menos la mitad de las fuentes **del nodo** contienen el mismo hecho, se lo considera consensuado;
- absoluta: si todas las fuentes **del nodo** contienen el mismo, se lo considera consensuado.

Si no se especifica un algoritmo de consenso, **dentro de la colección** todos los hechos se consideran consensuados y estarán disponibles en la navegación curada.

Es importante notar que, como consecuencia de la forma en que los algoritmos de consenso están definidos en los párrafos anteriores, el cálculo del consenso de cada hecho se realiza utilizando **información global**: se tienen en cuenta todas las fuentes del nodo. Sin embargo, el resultado de este cálculo en sí es **local a la colección**: se tiene en cuenta también el algoritmo de consenso de la colección. En otras palabras, una colección podría considerar consensuado a un cierto hecho, mientras que otra podría considerarlo no consensuado.

Por estos motivos, se considera que la ejecución de estos algoritmos **puede ser costosa**, por lo que es importante que no se ejecuten cada vez que ingresa un hecho, sino en horarios de baja carga en el sistema.

Requerimientos de dominio:

1. Como **persona visualizadora**, deseo poder seleccionar el modo de navegación de los hechos.
2. Como **persona administradora**, quiero asociar un algoritmo de consenso a una colección.

3. Cómo **persona visualizadora** del sistema, quiero poder buscar y navegar hechos en colecciones que tomen datos de múltiples fuentes.
4. Cómo **persona visualizadora** del sistema, quiero que la búsqueda y navegación de hechos de una colección siga siendo lo más rápida posible.
5. Cómo **persona contribuidora** del sistema, deseo que las solicitudes de eliminación aprobadas continúen teniendo efecto en las colecciones, aún cuando estás estén vinculadas a agregadores.
6. Cómo **persona encargada de la infraestructura** del sistema, quiero que cualquier calendarización sea realizada con planificación externa aprovechando las características de los sistemas operativos Linux.
7. Cómo **persona encargada de la infraestructura** del sistema, deseo que sea posible en todas o algunas colecciones evitar que cada búsqueda o navegación produzca llamadas a sistemas externos. Esto es particularmente importante cuando las colecciones tomen datos de agregadores.
8. Bonus: cómo **persona encargada de la infraestructura** del sistema, deseo que las fuentes dinámicas guarden su información periódicamente en archivos JSON, a modo de copia de seguridad (*backup*) en caso de que cualquiera de los servidores se detenga.

Entregables

1. **Modelo del Dominio**: diagrama de clases que contemple las funcionalidades requeridas.
2. **Justificaciones de Diseño**: Documento y Diagramas Complementarios.
3. **Implementación** de los requerimientos de esta entrega.
4. **Diagrama de despliegue** actualizado.
5. **Elección y definición detallada de un nuevo requerimiento** a criterio del equipo, en base al análisis y discusiones sobre las cuestiones de rendición de cuentas de la entrega anterior.